Deep Hedging Benchmark

Technical Report and Research Plan

Do learned hedging policies beat classical transaction-cost bands, and how much of any advantage survives a regime shift?

Field	Value
Author	Aditya Bhatia, Stevens Institute of Technology
Date	25 August 2026
Repository	github.com/Adi7710/deep-hedging-benchmark (private)
Stage	0 of 6 complete and audited
Test suite	49 passing, 7 skipped, 0 failing
Ladder	rungs 1 and 2 green; baseline half of rung 5 green
Target	arXiv preprint / thesis chapter

Status. This report documents completed work and plans work not yet begun. No neural network has been trained and no deep hedging result exists. Part I is a record of what is implemented and verified; Parts II and III are plans. Nothing here should be read as an empirical finding about deep hedging.

Contents

PART I. WHAT HAS BEEN BUILT

1. The problem

1.1 The situation

A dealer sells a European call and receives a premium. If the underlying rallies the dealer owes the difference between terminal price and strike. The position has bounded upside and unbounded downside, so it is hedged: the dealer holds a quantity of the underlying that offsets the option's exposure.

How much to hold is the entire question of this field. Black and Scholes answered it for an idealised market in 1973: hold Phi(d1) units, adjust continuously, and replication is exact. The option can be manufactured out of stock and cash, and residual risk is zero. That result is why listed option markets are the size they are.

1.2 Why the classical answer breaks

Black-Scholes requires continuous trading, zero transaction cost, constant known volatility, and continuous price paths. The first two fail together and fatally. A strategy that rebalances continuously incurs unbounded cumulative cost under any proportional cost rate, so the frictionless optimum is not merely inaccurate under frictions, it is inadmissible.

The real problem is a trade-off. Hedging more finely reduces replication variance and increases cost paid; hedging less finely does the reverse. Whalley and Wilmott (1997) solved this asymptotically for small proportional costs: the optimal policy is not to track the delta but to maintain a **no-transaction band** around it, trading only on exit from the band and only as far as the nearest boundary.

1.3 What deep hedging does

Buehler, Gonon, Teichmann and Wood (2019) replace derivation with optimisation. Rather than solving a Hamilton-Jacobi-Bellman equation for the optimal control, one parameterises the control by a neural network, simulates a large batch of price paths, evaluates the terminal P&L distribution under a convex risk measure, and differentiates that scalar with respect to the network parameters.

The method needs only the ability to **simulate and evaluate** a strategy, never the ability to **label** the correct action. That asymmetry is the whole methodological content. Evaluation is available wherever simulation is; labels exist only where a closed form already does. Deep hedging therefore extends immediately to stochastic volatility, jumps, incomplete markets, path-dependent claims and non-trivial cost structures, none of which have closed forms.

1.4 The gap this project addresses

Every published deep hedging method is evaluated in a setting of its authors' choosing. Simulator, cost rate, risk measure, rebalancing frequency and reported metric all differ simultaneously. Controls that would make numbers comparable, such as equal parameter count, equal training budget, multiple seeds with dispersion reported and a stated compute budget, are typically absent. A 2023 review states the consequence directly: the lack of a standardised testing dataset or universal benchmark makes it difficult to compare results across studies.

Worse, most comparisons are drawn against *naive delta hedging*, which rebalances at every date regardless of trade size. Under costs, any policy that trades less beats it. A demonstration that a learned policy beats delta hedging under costs is therefore weak evidence of learning. **The informative comparison is against Whalley-Wilmott**, and enforcing that is a central design commitment here.

1.5 The two research questions

- **Q1.** Under one fixed protocol, with the same simulators, cost models, risk measures, seeds and metrics, do deep hedging agents outperform classical transaction-cost-aware hedging?

- **Q2.** How much of any measured advantage survives a regime shift between the training and evaluation distributions?

The contribution is **methodological rigour and reproducibility**, not a new hedging algorithm. The design commits in advance to reporting null results, including cells where deep hedging ties or loses. Selective reporting of favourable cells is the specific failure this benchmark exists to correct, and a benchmark that succumbs to it has no value.

2. Architecture

Four components, built in dependency order. Each is verifiable before the next is added, which is what makes the correctness ladder meaningful rather than decorative.

```
     worlds/gbm.py         "what does the market do?"     -> price paths
            |
            v
     [ a strategy chooses delta ]     <- classical now, a neural network later
            |
            v
     pnl.py                "what did that earn?"          -> ONE NUMBER PER PATH
            |
            v
     [ a risk measure scores the distribution ]            <- Stage 2, not built

   standing aside as yardsticks:
     baselines/bs_delta.py          zero-cost ground truth   (is it RIGHT?)
     baselines/whalley_wilmott.py   the with-cost bar        (is it GOOD?)
```

The structural fact that organises everything: every component returns one number per path. Simulating 20,000 paths yields 20,000 P&Ls, and that *distribution* is the object of study. No individual number means anything on its own; every claim in the paper is a summary statistic of the distribution.

Module	Role	Language	Status
worlds/gbm.py	generates price paths	TensorFlow	complete
pnl.py	P&L accounting, single source of truth	TensorFlow	complete
baselines/bs_delta.py	closed-form price, delta, gamma	NumPy / SciPy	complete
baselines/whalley_wilmott.py	no-transaction band baseline	NumPy	complete
seeding.py	replicate-index hashing	stdlib + TF	complete
objectives/*	convex risk measures	TensorFlow	not started
agents/*	learned policies	Keras 3	not started
worlds/heston.py and others	alternative market models	TensorFlow	not started
evaluation/*	metrics and stress tests	NumPy	not started

Why the NumPy / TensorFlow split is deliberate

Reference values are implemented in NumPy and never enter a training graph. The test suite therefore validates TensorFlow-side code against machinery that **shares nothing with it**. Two independent implementations agreeing is evidence; one implementation agreeing with itself is not. Baselines are NumPy for the same reason, plus the practical one that the band rule is sequential and never differentiated.

3. The four components in detail

3.1 The world: worlds/gbm.py

Geometric Brownian motion models *percentage* changes as random rather than dollar changes. Both terms of the SDE carry S, so a 100 dollar stock and a 200 dollar stock move by the same proportion. That single assumption is why the solution involves an exponential.

```
dS = mu*S*dt + sigma*S*dW

exact solution, which is what is implemented:
  S_{i+1} = S_i * exp( (mu - sigma^2/2)*dt + sigma*sqrt(dt)*Z_i ),   Z_i ~ N(0,1)
```

The Ito correction, and why omitting it is not cosmetic

By Jensen's inequality E[exp(X)] is greater than exp(E[X]); for normal X the gap is exactly exp(variance/2). Without the -sigma^2/2 term the stock drifts at mu + sigma^2/2 rather than mu. The exponential is convex, so random shocks help more than they hurt, and that asymmetry accumulates as drift nobody asked for.

```
measured, 200k paths, mu = 0.05, T = 1:
  E[S_T] with the correction     105.13    theory 100*exp(0.05) = 105.13
  E[S_T] without it              107.25    wrong by 2%, far outside MC error
```

Exact solution rather than Euler

The Euler step is an approximation carrying O(dt) discretisation bias and can produce negative prices. The distinction matters more than accuracy alone, because **bias behaves differently from noise**. Monte Carlo noise shrinks as one over the square root of the path count and disappears with more sampling; discretisation bias does not shrink at all. The exact form is the true conditional law of GBM at any step size, so rung 1's residual is pure noise. With Euler, rung 1 would carry a small permanent discrepancy that presents as a pricing bug rather than a simulator bug, and costs an afternoon.

Vectorisation and reproducibility

All randomness is drawn in a single call. A generator's output depends on its call pattern, so one draw of shape (n_paths, n_steps) is a deterministic block, whereas a per-step loop consumes a different sequence and produces different paths from the same seed. Because bit-reproducibility is a claimed contribution, this is a correctness decision rather than a performance one. The log-price is a cumulative sum, so the implementation is cumsum, exp, and a concatenated s0 column. Concatenated rather than computed, so that column 0 is bit-exactly s0 rather than the output of exp(0).

3.2 The scorekeeper: pnl.py

The most important module in the project and deliberately the smallest. It is the single source of truth for P&L; nothing else computes it. The rationale is empirical: the dominant failure mode in reproductions of this literature is a bookkeeping error concealed inside a training loop, where it presents as a convergence problem and costs days. Isolating the arithmetic makes it testable independently of anything stochastic.

```
PL = p0 - Z + sum_{i=0}^{n-1} delta_i*(S_{i+1} - S_i)
           - sum_{i=0}^{n}   c*S_i*|delta_i - delta_{i-1}|

  delta_{-1} = 0   start flat
  delta_n    = 0   liquidate at maturity
```

This is **derived, not declared**. Starting from the self-financing condition, which states that rebalancing exchanges stock for cash and does not change wealth except for the fee incurred, one writes a recursion for the cash balance and telescopes it. The intermediate delta_i * S_i terms cancel pairwise and what survives is the discrete stochastic integral. Knowing the derivation is what permitted the correct extension to non-zero interest rates described in section 3.2.3.

Convention 1: predictability, and the shape contract

delta_i is chosen at time t_i knowing S_i but not S_{i+1}, then held across the interval, so it multiplies the increment over that interval. There are n+1 prices, n gaps, and n decisions. The terminal price is used to value a position but never to choose one.

Pairing delta_i with the preceding increment instead produces a look-ahead: a strategy trading on a move it has already observed. **No shape error is raised.** It presents as a hedge that performs implausibly well, which is why array shapes are treated as a contract rather than a convenience.

Convention 2: the cost sum runs to n, not n-1

```
the position carried through life:

   0  ->  delta_0  ->  delta_1  ->  ...  ->  delta_{n-1}  ->  0
   ^                                                        ^
   own nothing yet                     cannot walk away holding stock

   n decisions, n+1 transitions, every transition a real trade
```

Two arguments make the terminal term non-optional. The accounting argument is that terminal P&L is a cash quantity, whereas a share still held at maturity is a mark-to-market value; omitting the unwind asserts the position can be liquidated at mid. The incentive argument is sharper and is the reason it is emphasised: **the terminal cost is the only term penalising a large position at expiry**. Without it a trained policy can carry an unbounded hedge into maturity at no charge, improving the reported risk measure through a bookkeeping artefact. The strategy is not merely mismeasured but fictitious, and no internal consistency check detects it because the accounting remains self-consistent, merely wrong.

At fifty basis points on a call whose delta tends to one on in-the-money paths, the omitted charge is of order 0.50 on a spot of 100, a material fraction of a typical premium.

Convention 3: numeraire

The functional above was derived at zero interest rate, since only then does the cash account cancel. Rather than carry an explicit cash-account state machine, the benchmark is specified in **discounted (time-zero) units**. Two independent facts make this free.

- The discounted wealth of a self-financing strategy is the stochastic integral of delta against the *discounted* price, so the gains term carries through unchanged.
- A cost paid at t_i is c*S_i*|dq| in time-i money, so discounting gives c*S~_i*|dq|. **The cost term discounts by the same factor as the price it is levied on.** Nothing forced this; a flat per-trade fee would not have this property.

The functional is therefore form-invariant, and hedging_gains and transaction_costs take no rate argument at all. terminal_pnl is the sole site of discounting. The premium needs no adjustment because it is received at time zero and is already time-zero money, as is the Black-Scholes price it is usually set to. That consistency is why the change of numeraire costs nothing.

> **Stated boundary.** Collapsing the cash account into a single factor assumes one rate for both borrowing and lending. Funding spreads, collateral or asymmetric borrow and lend rates would break the collapse and require explicit cash flows. Recorded in the limitations section rather than discovered later. A second consequence: risk aversion is defined against a numeraire, and Whalley-Wilmott's is over terminal wealth while ours acts on discounted P&L. At zero rate the distinction vanishes; above zero it must be reconciled before any agent-versus-band comparison.

One implementation rule that will matter enormously

No conversion to NumPy anywhere in this module. In Stage 2 gradients flow from the risk measure back through terminal_pnl, back through every hedging decision, into the network weights. A NumPy round trip severs that chain and does so *silently*, yielding zero gradients and a network that appears to train but does not learn. A regression test now pins this: it computes a gradient through terminal_pnl and fails if it is None or identically zero.

3.3 The reference: baselines/bs_delta.py

```
d1    = ( ln(S/K) + (r + sigma^2/2)*tau ) / ( sigma*sqrt(tau) )
d2    = d1 - sigma*sqrt(tau)
C     = S*Phi(d1) - K*exp(-r*tau)*Phi(d2)
Delta = Phi(d1)
Gamma = phi(d1) / ( S*sigma*sqrt(tau) )
```

Written complete by design, since it is the yardstick everything else is measured against and needs to be right from day one rather than right eventually. Gamma appears because the Whalley-Wilmott band width depends on it.

Time to maturity is clipped to a tiny positive floor rather than allowed to reach zero. At expiry d1 is singular; the clip makes it diverge cleanly so that Phi saturates to one or zero, instead of producing a NaN that then propagates silently through an entire path.

3.4 The bar: baselines/whalley_wilmott.py

This module is the conceptual heart of the project's scepticism. Delta hedging rebalances at every step regardless of trade size, so under costs it bleeds, and any strategy trading less will beat it. Beating Whalley-Wilmott means something.

```
H = ( (3/2) * c * S * Gamma^2 * exp(-r*tau) / lambda )^(1/3)

        delta_BS + H   ------------------   edge
                          hold, do nothing
        delta_BS - H   ------------------   edge

  on exit, trade to the NEAREST EDGE, never back to the centre
```

Why the exponent is one third

Balance two competing effects. Cost paid scales as c/H, since a wider band means fewer boundary crossings. Risk carried scales as H squared, since a wider band means sitting further from the delta. Minimising the sum gives H cubed proportional to c. The cube root arises because risk grows quadratically in width while the saving grows only linearly, so the optimum moves reluctantly: five times the cost widens the band by only 1.71 times.

The three scalings, H as the cube root of cost, the two-thirds power of gamma, and the inverse cube root of risk aversion, are pinned by test to machine precision. They are not trivia. They are the **structural signature** a learned agent must reproduce, and an agent whose band fails to respond to cost as a cube root has not found the right policy shape however competitive its risk number appears.

The implementation insight

The band rule sounds like branching logic. It is not; it is a clip. Inside the band, clipping returns the held position unchanged, which is no trade. Outside, it returns the nearer bound, which is a trade to that edge. Nearest-edge behaviour is what clipping does by construction, so writing it this way makes the trade-to-the-centre bug **unrepresentable** rather than merely warned against.

This is also the first genuinely path-dependent component: step i depends on step i-1, so it cannot be a single vectorised expression. The loop runs over time and vectorises across paths, which is a direct structural preview of the Stage 2 training rollout.

The convention was measured, not asserted

```
20,000 paths, 50 steps, 50bp cost, 20 seeds, paths shared per seed

  comparison            mean       sd       t   consistent
  edge   vs delta    +0.1343   0.0446   13.46      20/20
  centre vs delta    +0.0354   0.0327    4.84      17/20
  edge   vs centre   +0.0989   0.0405   10.93      20/20

  the ratio centre/edge is NOT quotable: mean 24%, sd 27%,
  range spanning zero. Report the difference in levels.
```

A benchmark implementing the centre rule would report a Whalley-Wilmott baseline nearly indistinguishable from naive delta hedging, and would therefore overstate any learned policy's advantage by roughly an order of magnitude. That is precisely the failure this project exists to prevent, so the convention is now pinned by an acceptance test rather than by a comment.

4. Verification: the correctness ladder

The deepest idea in Stage 0 is not in any single file. It is the verification strategy: each rung is meaningful only if its predecessors hold, and no learned result is interpreted until the layers beneath it are verified.

Rung	Test	Gates	Status
1	Monte Carlo call price matches closed form	simulator and pricing	GREEN
2	Zero-cost fine-grid delta hedge, P&L std to zero	P&L accounting	GREEN, 13/13
3	Heston reproduces characteristic-function prices	stochastic vol simulator	not started
4	Learned agent recovers Phi(d1) at zero cost	THE HEADLINE GATE	not started
5	Learned band approximates Whalley-Wilmott	frictions	baseline half GREEN
6	Every experiment bit-reproducible from config and seed	the benchmark claim	not started

Why rung 2 is the load-bearing one

A short call, delta hedged at zero cost with the premium set to the Black-Scholes price, must produce terminal P&L concentrated near zero with residual dispersion scaling as the inverse square root of the step count. The test is strong because it is **joint**: it requires the simulator, the Black-Scholes reference and the accounting to be mutually consistent.

```
n_steps      mean PL     std PL   std*sqrt(n)
     10      -0.0039     2.1223         6.711
     40      -0.0093     1.0827         6.847
    160      -0.0010     0.5457         6.903
    640      -0.0023     0.2782         7.037

the last column is roughly constant -- that IS the 1/sqrt(n) law
```

Any one of the three components could be wrong in isolation. All three being wrong *in a way that still produces the correct convergence rate* is very unlikely. This is stronger evidence than three separate unit tests would provide, and it is the entire justification for building the classical foundation before touching a neural network.

5. Defects found and corrected

Four defects were found during Stage 0. Three were specification errors in the project's own design documents, and all three biased results *toward* deep hedging. The fourth was found by accident during an audit and is the most serious.

5.1 Band rebalancing target

Two design documents specified trading back to the Black-Scholes delta on band exit, while the implementation specified trading to the nearest boundary. The implementation was correct: the optimal policy under proportional costs is a singular control that executes the minimal trade returning the state to the no-transaction region. The error costs 0.099 of CVaR improvement, consistently across all twenty seeds tested (t = 10.9), as measured in section 3.4.

5.2 Cost-sum upper limit

The paper plan wrote the cost sum with upper limit n-1, omitting terminal liquidation, inconsistent with both the problem statement and the implementation. This is exactly the error identified as material in section 3.2.

5.3 The interest-rate gap

The P&L functional was derived at zero rate while the Whalley-Wilmott band carries a discount factor, reintroducing the rate at the final Stage 0 component. Resolved by the change of numeraire described in section 3.2.3.

5.4 Seed derivation: the serious one

Bit-reproducibility from configuration and seed is a claimed contribution, and reporting dispersion across replicate seeds is one of the controls this project criticises others for omitting. Both rest on an assumption that is easy to make silently and false in practice: that consecutive replicate indices, passed directly to the random number generator, yield independent streams.

```
pricing an ATM call, truth 7.96557, 200k paths, seeds 0..19

                       mean       bias        cross-seed dispersion
  from_seed(0..19)   7.99070   +9.3 SE            0.012
  hashed seeds       7.95994   -0.7 SE            0.034
  analytic SE                                     0.0296

  20/20 replicates above truth with raw seeds; 7/20 with hashed
```

The mechanism is visible in the underlying draws. Sample means of 200,000 standard normals from seeds zero through five were -0.00204 repeated to five decimal places, where independent streams should scatter with standard deviation 0.00224. That systematic offset propagates into realised volatility, roughly 0.2004 against an intended 0.2000, and at an at-the-money vega near 39.7, five basis points of volatility is two cents of option price. That accounts for the observed bias exactly.

Understating dispersion by a factor of two and a half is the more serious half. It does not make the code incorrect; it makes the uncertainty quantification dishonest, and would report comparisons as significant that the evidence does not support. That is a failure of the same kind as, and worse in degree than, the ones this benchmark is intended to correct.

Replicate indices are now hashed with SHA-256 before reaching TensorFlow. SHA-256 rather than Python's built-in hash, which is salted per process unless an environment variable is pinned and would itself break reproducibility across runs. A stream label is mixed into the digest, so disjointness between training and evaluation randomness holds by construction rather than by an additive offset a caller can omit.

> **Why this is reported as a finding rather than a fix.** It is invisible in any single run, it is not detected by a same-seed reproducibility check, and it is a plausible unexamined defect in published work reporting seed-averaged error bars. The property is now enforced by test: observed dispersion across sixteen

replicates must fall within 0.6 and 1.7 times the analytic standard error. Unhashed seeding scores 0.34 and fails, so the test has demonstrated power.

5.5 Audit items cleared

Stated explicitly because two were suspected and both were wrong. Float32 precision was checked by comparing float32 and float64 path construction driven by *identical* normal draws: prices agreed to six decimal places with a maximum relative path error of 1.9e-6 at 2520 steps. Accumulation in the mean over 200,000 paths showed zero difference against float64. The initial measurement that suggested a precision problem had conflated precision with Monte Carlo noise, because the two dtypes consume different random draws from the same seed.

6. Current status

Metric	Value
Tests	49 passing, 7 skipped, 0 failing
Skipped	rungs 3, 4, 6 and the learned half of rung 5, retained as executable specifications
Modules complete	pnl.py, worlds/gbm.py, baselines/bs_delta.py, baselines/whalley_wilmott.py, seeding.py
Ladder green	rungs 1 and 2; baseline half of rung 5
Not implemented	risk measures, all agents, Heston and other worlds, Zakamouline, evaluation layer, config runner
Data access	WRDS / OptionMetrics confirmed; not yet used

Therefore. The classical foundation is complete, audited and verified. No learned policy has been trained, and no number produced so far is a result about deep hedging.

PART II. HOW NEURAL NETWORKS ENTER

7. The conceptual shift

Everything built so far computes delta from a **formula**: either Phi(d1) or the band rule. Everything from here replaces the formula with a **parameterised policy** that has never been told what a delta is.

Element	In this problem
State	what is known at time t_i: the clock, the price, and the position currently held
Action	delta_i, the quantity to hold over the next interval
Dynamics	the price evolves stochastically; the position evolves as chosen
Reward	NONE per step. One number at the very end: terminal P&L
Objective	minimise a convex risk measure of that terminal P&L

The reward row deserves emphasis. There is no signal indicating whether any individual trade was good. Feedback arrives once, at maturity, after every decision has already been made. That is what makes this a stochastic control problem rather than a prediction problem.

Classical control *solves* for the optimal policy analytically. That works under GBM at zero cost, where the answer is Phi(d1), and becomes intractable as soon as costs, stochastic volatility or jumps are introduced. Deep hedging does not solve; it **parameterises and optimises**.

```
1. simulate a batch of price paths
2. walk each path, asking the network for delta at every step
3. compute terminal P&L for each path            <- pnl.py, already built
4. score the distribution with a risk measure    <- one scalar
5. differentiate that scalar w.r.t. the weights
6. take an optimiser step and repeat
```

That is the entire model. Six lines. The network is never shown a correct answer, because none exists in the general case; it is told that the P&L distribution it produced was this risky, and it adjusts.

8. Why this cannot be supervised learning

Under GBM at zero cost the optimal control is known in closed form, so it is reasonable to ask why the policy is not simply regressed onto Phi(d1). Four reasons, in increasing order of consequence.

8.1 Labels exist only where the answer is already known

```
zero costs           -> Phi(d1)                   known
+ proportional cost  -> a no-transaction band     ASYMPTOTIC only
+ stochastic vol     -> incomplete market         no closed form
+ jumps              -> not perfectly hedgeable   no closed form
+ discrete steps     -> not exactly Phi(d1)       no closed form
```

A supervised policy can reproduce only what is already available. Deep hedging requires the ability to evaluate a strategy, which is available wherever simulation is. Everywhere new is the entire reason the method exists.

8.2 The surrogate loss is not the objective

Squared error on the control weights all deviations equally. A delta error of fixed size is nearly costless when gamma is small and maturity distant, and expensive at the money near expiry, where gamma is enormous and the underlying may gap through the strike. Mean squared error is a pointwise loss on the *control*; the true objective is a functional of the induced P&L *distribution*. They do not share a minimiser away from the idealised case.

8.3 A pointwise label cannot represent the optimal policy under costs

```
zero costs:   optimal delta = f(time, S)
              rebalancing is free, so where you are now is irrelevant

with costs:   optimal delta = f(time, S, delta_prev)
              if you are already close, DO NOT TRADE -> this is the band
```

Once trading is costly the current position enters the state. The label Phi(d1) contains no dependence on the previous position, so a policy supervised on it **cannot express a no-transaction band even in principle**. The band is the object of interest in this entire literature.

8.4 It destroys the verification protocol

Rung 4 requires that an agent trained at zero cost recovers Phi(d1). That test has diagnostic power precisely because the delta is never shown to the network: the policy arrives at Phi(d1) by numerical optimisation of a risk functional, along a route entirely independent of the Black-Scholes derivation. Agreement is therefore joint evidence for the simulator, the accounting and the training loop simultaneously. Supervising on Phi(d1) reduces the test to confirming that a regression fits its own labels.

A subtlety justifying the approximation sign in rung 4. Phi(d1) is optimal in continuous time. At a finite number of steps the truly risk-minimising discrete position differs from it, and the difference depends on the risk measure chosen. A correctly trained agent solves the discrete problem actually posed, for which no closed form exists, and should therefore approach but not equal Phi(d1), converging as the step count grows.

9. The five missing pieces

Three components must be built, and they are ordered so that each fails loudly and in isolation.

9.1 The judge: convex risk measures

Turns a distribution of 20,000 P&Ls into a single score. **No neural network is involved**, so this is a pure function testable by hand exactly as pnl.py was, and it is therefore built first.

```
entropic (exponential utility), risk aversion lambda > 0:
    rho(X) = (1/lambda) * log E[ exp(-lambda*X) ]
    implement with reduce_logsumexp; log(mean(exp(.))) overflows

CVaR at level alpha, Rockafellar-Uryasev form:
    rho(X) = min over w of { w + (1/(1-alpha)) * E[ (-X - w)^+ ] }
    w is a TRAINABLE SCALAR optimised jointly with the weights,
    not an inner optimisation loop

mean-variance (not coherent; included for comparability):
    rho(X) = -E[X] + (lambda/2)*Var(X)
```

The first conceptual question this settles is why the objective is not simply to maximise expected P&L. Maximising the mean rewards taking unhedged directional risk, since the expected cost of a hedge is positive while its benefit is entirely in variance reduction. A risk measure is what makes hedging rational rather than value-destroying.

Cash-invariance. All three satisfy the property that adding a constant to the P&L shifts the risk measure by the negative of that constant. The premium therefore translates the objective and cannot change the optimal policy. This matters operationally: when a learned policy appears indifferent to the premium, that is correct behaviour rather than a bug, and it is a common source of wasted debugging.

9.2 The tool: GradientTape, learned on a toy problem

TensorFlow's mechanism for computing derivatives. This will be learned on something trivial, such as locating the minimum of a quadratic, and explicitly **not** on the hedging problem. Learning a new tool and a new problem simultaneously means that when something breaks, it is not clear which one broke.

9.3 The agent

```
input :  (time_to_maturity, spot or moneyness, delta_prev)
output:  delta, the quantity to hold now

  hidden layers: [32, 32], ReLU
  output activation: NONE
```

Weight sharing across time is the point most often misunderstood. There is one network, used at every timestep, not one network per step. The time index is an *input* to the network rather than a selector among models. This is what allows a policy trained on a fixed grid to be evaluated at arbitrary times and moneyness, which is exactly what the rung 4 overlay plot requires.

The output activation is deliberately unconstrained. A sigmoid would force the output into the unit interval, which happens to be the correct range for a call delta. Supplying that range would weaken rung 4 considerably: the gate is meaningful because the network **finds** the interval, not because it was handed it. The reference configuration records this decision explicitly.

delta_prev must be an input. Without it the policy cannot express a band, for the reason given in section 8.3, and the entire frictions half of the project becomes unreachable.

9.4 The rollout

Walk down each price path, query the agent at every step, and assemble the answers into the standard position array. Structurally this is the loop already written in the band baseline: vectorise across paths, iterate

over time, carry the previous position forward. The only difference is that the rule inside the loop is a network rather than a clip.

9.5 The trainer

Backpropagation through time. The decision at step zero affects terminal P&L through every subsequent price increment *and* through the cost of every subsequent trade, so the gradient accumulates along the entire chain of decisions. This is structurally identical to training a recurrent network, and it is why model.fit cannot express the problem: there are no independent examples to sum a loss over.

```
with tf.GradientTape() as tape:
    delta = rollout(agent, spot)          # ALL of it inside the tape
    pnl   = terminal_pnl(spot, delta, payoff, cost_rate, premium)
    loss  = risk_measure(pnl)             # one scalar
grads = tape.gradient(loss, agent.trainable_variables)
optimizer.apply_gradients(zip(grads, agent.trainable_variables))
```

10. The specific ways this goes wrong

Recorded in advance, because the rung 4 failure checklist is only useful if the failure modes are understood before they occur.

Failure	Symptom	Why it happens
Rollout outside the tape	gradients are None; nothing trains	the tape only records operations executed inside its context
delta_prev detached from the graph	trains, but never learns a band	the path from today's trade to tomorrow's cost is cut, so the policy cannot see that trading has future consequences
Inputs not normalised	slow or stalled convergence	spot near 100 and time near 1 differ by two orders of magnitude, so gradients are badly scaled
Constrained output activation	rung 4 passes but proves little	the correct range was supplied rather than discovered
log(mean(exp(.))) for entropic risk	NaN or inf loss	the exponential overflows; reduce_logsumexp is numerically stable
A NumPy conversion inside pnl.py	zero gradients, silently	the tape is severed; now pinned by a regression test

Every one of these produces plausible output rather than an exception, which is the recurring theme of this project and the reason for the verification-first structure.

11. The gate: rung 4

Train the feedforward agent under GBM at zero transaction cost using the reference configuration. The learned policy must reproduce Phi(d1).

```
acceptance:
  moneyness S/K in [0.8, 1.2], time to maturity in [0.1, 1.0]
  mean absolute deviation from bs_delta below 0.05
  no single point off by more than 0.15

  plus the overlay plot -- far more convincing to a reader
  than the numeric threshold, and it belongs in the paper
```

If this fails, everything downstream is meaningless and work stops until it passes. The diagnostic order is fixed: confirm rungs 1 and 2 are still green, then that the rollout is inside the tape, then that delta_prev is not detached, then input normalisation, then that the entropic loss uses the stable form.

A second Stage 2 gate follows: the indifference price, obtained from two training runs with and without the claim, must approximate the Black-Scholes price of 7.9656 in the frictionless limit. That is an economic check rather than a numerical one, and it verifies that the learned policy is not merely shaped correctly but priced correctly.

PART III. RESEARCH PLAN

12. Stage roadmap

Stage	Content	Gate	Compute
0	Classical foundation: GBM, MC pricing, P&L, delta and band hedging	rungs 1 and 2 green	CPU
1	GradientTape, custom training loops, Keras model subclassing	toy optimisation converges	CPU
2	Vanilla deep hedging, feedforward agent, entropic objective	RUNG 4: learned delta overlays Phi(d1)	CPU
3	Frictions, Heston, recurrent agent, CVaR	rungs 3 and 5 green	GPU (Colab)
4	Freeze the benchmark protocol; publish configs	rung 6: bit-reproducible	GPU
5	Re-implement published variants, run the full grid, report nulls	every cell populated	many GPU-hours
6	Writeup, figures, arXiv submission	-	CPU

Stage 0 is complete. Stages 1 and 2 run on the local machine. From Stage 3 onward a GPU is required and Google Colab is the plan; the local AMD integrated GPU is not a supported TensorFlow path.

13. Stages 1 and 2 in detail

Step	Deliverable	Testable without	Est.
1	objectives/entropic.py: entropic risk measure	any neural network	1 session
2	GradientTape on a toy quadratic	the hedging problem	1 session
3	agents/feedforward.py: the policy network	training	1 session
4	rollout: walk paths, query agent, assemble delta	training	1 session
5	the training loop	-	1-2 sessions
6	rung 4: overlay plot and acceptance test	-	1 session
7	indifference price gate	-	1 session

The ordering is chosen so that by the time step 5 is reached, the only thing that can be wrong is step 5. A broken risk measure is caught at step 1 with no network present; a misunderstood tape is caught at step 2 on a quadratic; wrong shapes are caught at step 3 before any training; a wrong rollout is caught at step 4 by feeding it a known analytic policy and comparing against the delta-hedge path already verified in rung 2. This is the same discipline that made Stage 0's ladder meaningful.

Step 4 has a free and powerful test. Replace the network with a function returning Phi(d1) and the rollout must reproduce, to floating-point tolerance, the position array that delta_hedge_positions already produces. That validates the loop mechanics completely, using machinery verified in rung 2, before any learning is involved.

14. Stages 3 to 6

14.1 Stage 3: frictions and richer worlds

- **Heston stochastic volatility**, validated against semi-analytic characteristic-function prices (rung 3). Must use full truncation for the variance process and must be tested with a parameter set that violates the Feller condition, since that is the regime where naive schemes go negative.
- **Recurrent agent** following Carbonneau (2020), with an LSTM cell stepped manually so the hidden state carries path history rather than relying on a hand-specified state vector.
- **CVaR and mean-variance objectives**, enabling the risk-measure-sensitivity question.
- **Rung 5**: the learned band must correlate above 0.8 with the analytic Whalley-Wilmott width and, more importantly, share its shape, peaking where gamma peaks. Shape agreement matters more than level agreement, since the analytic band is asymptotic in small cost and an exact level match at realistic cost levels would be suspicious.

14.2 Stage 4: freezing the protocol

The contribution itself. Everything is fixed and published as configuration files: four worlds, three cost levels plus a fixed-cost cell, three risk measures, a metric set, and the controls the literature usually omits, namely equal parameter count, equal training budget, multiple seeds with dispersion, and a stated compute budget.

Metric	Why it is included
CVaR-95 of terminal P&L	the tail is what hedging exists to control
Standard deviation of P&L	comparability with older literature
Turnover	tests whether an advantage is merely more trading
Total cost paid	separates gross skill from net outcome
Indifference price vs Black-Scholes	economic interpretation
Degradation ratio under train/test mismatch	the regime-fragility result

Blocking item for Stage 4. transaction_costs currently hardcodes proportional costs and cannot express the fixed-plus-proportional model. That is the *non-convex* cell of the grid, the one that tests directly whether gradient-based deep hedging fails without convexity as arXiv:2510.01874 claims. The signature of the single-source-of-truth module must change before the protocol is frozen, not after results exist.

14.3 Stage 5: the grid, and the null-result commitment

Worlds crossed with costs, risk measures and agents. All of it is reported, including cells where deep hedging ties or loses to the classical band. This is stated in advance because the incentive to quietly drop unflattering cells is precisely the failure the paper criticises.

Regime fragility is the section that turns a replication into a benchmark result. Train on one world, evaluate on another, report the degradation ratio for every method. The 2026 paper on what deep hedging actually learns reports that learned policies function largely as delta corrections and fail outside the training distribution; quantifying that across the full method set rather than one model is the contribution. If the benchmark framing is pre-empted, this section is the fallback standalone paper.

15. Real data: WRDS and OptionMetrics

Institutional access is confirmed. The single most important decision is instrument choice.

Use SPX index options, not single-name equity options. OptionMetrics covers both, but US single-name equity options are *American*. Early exercise makes the payoff date a stopping time rather than a fixed maturity, which breaks the P&L functional in section 3.2. SPX index options are European and cash-settled, matching both the fixed-maturity and cash-settlement conventions already built in. Choosing wrongly would silently change the problem being solved.

Stage	Use	Value
4	Cost calibration from realised SPX bid-ask spreads	turns the cost rate from an arbitrary 50bp into an empirical number; removes an obvious reviewer objection to the entire cost grid
5	Heston calibration to a real implied-volatility surface	answers whether simulator parameters were cherry-picked
5+	Hedge realised SPX paths with simulator-trained policies	the strongest form of the fragility result: simulator-to-reality rather than simulator-to-simulator

The third changes the paper. The fragility section currently asks whether a policy trained on Heston survives regime-switching, and both of those distributions were invented here. Asking whether it survives the actual market is a qualitatively harder test, because reality is in none of the simulator families.

Two constraints. OptionMetrics is not redistributable, so no raw data enters the repository and the retrieval script plus its date is the reproducibility artefact. And a code change is implied: SPX pays dividends, so the Black-Scholes functions need a continuous dividend yield, changing d1 and multiplying delta by a discount factor. Not needed for the simulated benchmark, where the yield is zero, but required before any real-data section.

16. Paper structure

Section	Content	Ready after
1	Introduction: the comparability gap	now
2	Related work: Buehler 2019 through the 2025-26 wave	now
3	Problem setup: market, claim, P&L, risk measures, numeraire	Stage 0, DONE
4	Baselines: Black-Scholes delta, Whalley-Wilmott, Zakamouline	Stage 0, mostly done
5	Benchmark protocol	Stage 4
6	Agents: feedforward, recurrent, band, robust	Stages 2, 3, 5
7	Results: the full grid, including null results	Stage 5
8	Regime fragility	Stage 5
9	Limitations	-

A ten-page methods draft covering sections 1 through 4 and the verification protocol already exists in the repository, with an explicit status section stating that no empirical results have been produced.

17. Risks and mitigations

Risk	Mitigation
Rung 4 does not pass and the agent fails to recover Phi(d1)	This is the designed stopping point. The diagnostic order in section 11 is fixed in advance. Stage 0 being audited means the fault must lie in the training loop, which is a small and enumerable surface.
The benchmark framing is pre-empted by another group	The regime-fragility section stands alone as a focused paper. Literature searches are scheduled before Stage 4 and again before Stage 6.
Seed variance exceeds the effect sizes under study	Dispersion is reported rather than point estimates, and comparisons that do not survive seed variation are reported as inconclusive rather than as wins. The seeding audit in section 5.4 is what makes this claim credible.
A re-implemented method underperforms its published version	Each is validated against its source paper's headline number before entering the grid; any that cannot be validated is reported as unvalidated rather than as a loss.
Compute budget constrains hyperparameter search	The budget is stated explicitly as a limitation so the constraint is legible rather than hidden.
Real-data work expands scope uncontrollably	WRDS use is deliberately deferred and bounded to three specific tasks. The project is tractable because there is no dataset to acquire; access expands what the paper can claim, not what must be built.

18. Immediate next actions

#	Action	Blocking?
1	Implement objectives/entropic.py with reduce_logsumexp; test by hand	no
2	Learn GradientTape on a toy quadratic	no
3	Implement FeedforwardAgent; verify shapes only	no
4	Implement the rollout; validate against delta_hedge_positions	no
5	Write the training loop; run the reference configuration	no
6	RUNG 4: overlay plot and acceptance test	gates everything downstream

#	Action	Blocking?
7	Fix the cost-model interface before Stage 4 freezes the protocol	gates Stage 4

The single most important upcoming artefact is the rung 4 overlay: the learned hedge ratio plotted against Phi(d1) across moneyness and time to maturity. If those curves lie on top of one another, a network given nothing but a simulator, a P&L function and a risk score has rediscovered the Black-Scholes delta by a route entirely independent of the original derivation. That plot is simultaneously the project's correctness gate and the most convincing single figure available to a reader.